

# ORIE 5135 – Project

Andrea Lodi

Spring 2026

A university needs to schedule a set of  $m$  exams ( $i = 1, \dots, m$ ) into a set of  $n$  available time slots ( $j = 1, \dots, n$ ). Each exam requires a certain number of proctors: exam  $i$  needs  $r_i \in \mathbb{Z}_+$  proctors. In each time slot the university can deploy at most  $R$  proctors in total across all exams running simultaneously. We assume  $r_i \leq R$  for all  $i = 1, \dots, m$ .

Multiple exams can be administered simultaneously during the same time slot (in different rooms), provided their total proctor requirement does not exceed  $R$ . However, two exams that share at least one enrolled student **cannot** be scheduled in the same time slot, because every student must be able to take all of their exams.

The student-overlap information is encoded in a *conflict graph*  $G = (V, E)$  where  $V = \{1, \dots, m\}$  is the set of exams and an edge  $(i, i') \in E$  indicates that exams  $i$  and  $i'$  have at least one student in common.

The goal is to schedule every exam into exactly one time slot so that:

- the total proctor requirement of exams in each time slot does not exceed  $R$ ,
- no two exams connected by an edge in  $G$  are assigned to the same time slot, and
- the number of used time slots is minimized.

## Questions

1. Propose a “natural” (descriptive) Integer Linear Programming (ILP) formulation for the problem and implement it in code. In particular:
  - (a) Write the formulation and implement it in code.
  - (b) The natural formulation contains constraints that encode the conflict structure of the graph  $G$ . These constraints can be strengthened using the notion of *clique* in  $G$ , as discussed in class for the Stable Set problem. In your writeup, describe how to perform this strengthening. Then, implement the strengthened formulation in code.
2. Propose an extended formulation of the problem with a variable for each feasible exam grouping within a single time slot. A subset  $S \subseteq \{1, \dots, m\}$  is a feasible grouping if and only if

- $\sum_{i \in S} r_i \leq R$ , and
- $S$  is a *stable set* (independent set) in  $G$ , i.e., no two exams in  $S$  share a student.

- (a) Implement this formulation in code, assuming that all feasible groupings can be enumerated.

- (b) Discuss different choices of extended formulations, i.e. set covering vs. set partitioning.
3. Solve the ILP formulations on the two provided datasets as described below.

The instances are provided as JSON files. Each file contains the following fields: **m** (number of exams), **n** (number of available time slots), **R** (total proctor budget per slot), **r** (list of proctor requirements  $r_1, \dots, r_m$ ), and **edges** (list of conflicting exam pairs  $[i, i']$ ).

- (a) **Dataset 1** (time limit: 120 seconds). This dataset contains instances of increasing size. Solve all three formulations (Questions 1a, 1b, and 2) on every instance. Collect and report data on

- the LP relaxation value for each formulation,
- the number of branch-and-bound nodes explored, and
- the wall-clock solve time (as reported by the solver) and whether the instance was solved to optimality within the time limit.

Compare the three formulations based on your results. In particular: (i) compare the LP relaxation values of Formulations 1a, 1b, and 2 — does clique strengthening (Question 1b) improve the LP bound over the natural formulation 1a? Does the extended formulation (Question 2) provide a tighter LP relaxation? Report and discuss what you observe. (ii) Compare the branch-and-bound node counts and solve times. Do formulations with equal LP relaxation values always explore a similar number of nodes and require similar computing time? What does your data suggest about the relationship between LP bound quality and branch-and-bound performance? (iii) As instance sizes grow, how does the computational effort (node counts and solve times) evolve for each formulation? Does the rate of growth differ across the three formulations?

- (b) **Dataset 2** (time limit: 600 seconds). This dataset contains larger instances for which enumerating all feasible groupings is *not* practical. Solve only the natural formulation (Question 1a) and the clique-strengthened formulation (Question 1b) on every instance. Collect and report the same data as in part (a). Discuss the computational limits of these two formulations as instance sizes grow, and explain why the extended formulation is not applicable to these instances.

## Requirements and Deliverables

- All implementations must be in either Julia (using JuMP and Gurobi) or Python (using gurobipy). Your code must read instances directly from the provided JSON files.
- **Implementation:** implement the formulations of Questions 1 and 2a yourself, without using any provided starter code. Your implementation should be able to solve any instance from the provided datasets.
- **Writeup:** a PDF file containing the problem formulations of Questions 1 and 2, and the answers to Questions 2b and 3 (including all tables and discussion).
- **Submission:** provide a link to a GitHub (or GitLab) repository containing both your implementation and your writeup. If you prefer not to use a public repository, share a private repository with the course staff. Alternatively, a zip file is accepted as a fallback.